

Funzioni

Metodi

Come dividere un programma Java in blocchi riutilizzabili.

Cosa è un metodo

Un metodo è un blocco di codice con un nome.

Lo scrivi una volta e puoi richiamarlo quando serve.

Pensa a un metodo come a una piccola ricetta: invece di riscrivere ogni passaggio, dai un nome all'azione.

Un primo metodo

```
public class Saluti {  
    public static void saluta() {  
        System.out.println("Ciao!");  
    }  
  
    public static void main(String[] args) {  
        saluta();  
        saluta();  
    }  
}
```

Output:

```
Ciao!  
Ciao!
```

Il metodo `saluta` viene definito una volta e chiamato due volte.

Definire e chiamare

Questa parte definisce il metodo:

```
public static void saluta() {  
    System.out.println("Ciao!");  
}
```

Questa parte lo chiama:

```
saluta();
```

Definire un metodo non lo esegue automaticamente. Viene eseguito quando lo chiami.

Perche usare metodi

I metodi aiutano a:

- evitare ripetizioni
- dare un nome a un'azione
- dividere un programma in pezzi piu piccoli
- rendere il codice piu facile da leggere

Senza metodo:

```
System.out.println("-----");  
System.out.println("Prodotti");  
System.out.println("-----");  
System.out.println("Totale");  
System.out.println("-----");
```

Con metodo:

```
public static void separatore() {  
    System.out.println("-----");  
}
```

Poi:

```
separatore();  
System.out.println("Prodotti");  
separatore();  
System.out.println("Totale");  
separatore();
```

Metodi statici nei primi programmi

Nei primi esempi useremo spesso:

```
public static void nomeMetodo()
```

`static` permette di chiamare il metodo direttamente da `main`, senza creare oggetti.

Piu avanti, nella programmazione a oggetti, vedremo metodi legati agli oggetti.

Nomi chiari

Il nome del metodo dovrebbe dire cosa fa.

```
public static void stampaTitolo() {  
    System.out.println("Lista della spesa");  
}
```

`stampaTitolo` e piu chiaro di `faiCosa`.

In Java i metodi usano di solito il camelCase:

```
calcolaTotale()
mostraMessaggio()
leggiPrezzo()
```

Esempio completo

```
public class Scontrino {
    public static void stampaSeparatore() {
        System.out.println("-----");
    }

    public static void main(String[] args) {
        stampaSeparatore();
        System.out.println("Pane");
        System.out.println("Latte");
        stampaSeparatore();
        System.out.println("Totale: 3.20 euro");
        stampaSeparatore();
    }
}
```

Un metodo non deve fare tutto. Di solito è meglio che abbia un compito preciso e piccolo.

Overloading

Come definire più metodi con lo stesso nome ma parametri diversi.

Lo stesso nome, parametri diversi

In Java puoi avere più metodi con lo stesso nome, se hanno parametri diversi.

Questa tecnica si chiama **overloading**.

```
public static void saluta() {  
    System.out.println("Ciao!");  
}  
  
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome + "!");  
}
```

Java sceglie quale metodo chiamare guardando gli argomenti.

Un primo esempio completo

```
public class Saluti {  
    public static void saluta() {  
        System.out.println("Ciao!");  
    }  
  
    public static void saluta(String nome) {  
        System.out.println("Ciao, " + nome + "!");  
    }  
  
    public static void main(String[] args) {  
        saluta();  
        saluta("Luca");  
    }  
}
```

Output:

```
Ciao!  
Ciao, Luca!
```

La prima chiamata non passa argomenti. La seconda passa una `String`.

La firma del metodo

La **firma** di un metodo è composta da:

- nome del metodo
- tipi dei parametri
- ordine dei parametri

Questi metodi hanno firme diverse:

```
public static void stampa(String testo) {  
    System.out.println(testo);  
}  
  
public static void stampa(int numero) {  
    System.out.println(numero);  
}
```

Uno riceve una `String`, l'altro un `int`.

Il valore di ritorno non basta

Non puoi distinguere due metodi solo dal tipo di ritorno.

Questo codice è sbagliato:

```
public static int calcola() {  
    return 1;  
}  
  
public static double calcola() {  
    return 1.0;  
}
```

Per Java le chiamate sarebbero ambigue:

```
calcola();
```

Quale dei due dovrebbe scegliere?

Parametri in ordine diverso

L'ordine conta.

```
public static void mostra(String nome, int eta) {  
    System.out.println(nome + ", " + eta);  
}  
  
public static void mostra(int eta, String nome) {  
    System.out.println(nome + ", " + eta);  
}
```

Questi due metodi sono diversi per Java.

Pero non sempre è una buona idea: chi legge potrebbe confondersi. Usa l'overloading solo quando rende il codice più chiaro.

Esempio: calcolare il totale

```
public class Totali {  
    public static double totale(double prezzo, int quantita) {  
        return prezzo * quantita;  
    }  
  
    public static double totale(double prezzo, int quantita, double  
spedizione) {  
        return prezzo * quantita + spedizione;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(totale(2.50, 4));  
        System.out.println(totale(2.50, 4, 3.90));  
    }  
}
```

Output:

```
10.0  
13.9
```

Il nome `totale` resta lo stesso perché l'idea è la stessa. Cambiano solo i dati disponibili.

Quando usarlo

L'overloading è utile quando:

- i metodi fanno la stessa azione generale
- cambiano solo i parametri disponibili
- il nome comune aiuta a leggere meglio

Evitalo se i metodi fanno cose davvero diverse. In quel caso scegli nomi diversi.

Parametri e valori di ritorno

Come passare dati a un metodo e ricevere un risultato.

Rendere un metodo più flessibile

Un metodo senza parametri fa sempre la stessa cosa.

```
public static void saluta() {  
    System.out.println("Ciao, Luca!");  
}
```

Se vogliamo salutare nomi diversi, usiamo un **parametro**.

Parametri

```
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome + "!");  
}
```

`String nome` è un parametro: una variabile che il metodo riceve quando viene chiamato.

```
saluta("Luca");  
saluta("Sara");
```

Output:

```
Ciao, Luca!  
Ciao, Sara!
```

I valori passati al metodo si chiamano **argomenti**.

Piu parametri

Un metodo puo ricevere piu parametri.

```
public static void mostraProdotto(String nome, double prezzo) {  
    System.out.println(nome + ": " + prezzo + " euro");  
}
```

Chiamata:

```
mostraProdotto("Pane", 1.50);  
mostraProdotto("Latte", 1.20);
```

Output:

```
Pane: 1.5 euro  
Latte: 1.2 euro
```

L'ordine degli argomenti deve corrispondere all'ordine dei parametri.

void

Finora abbiamo usato `void` .

```
public static void saluta(String nome)
```

`void` significa: questo metodo non restituisce un valore.

Il metodo fa qualcosa, per esempio stampa un messaggio, ma non consegna un risultato a chi lo chiama.

return

Se un metodo deve produrre un risultato, devi indicare il tipo del valore restituito.

```
public static double calcolaTotale(double prezzo, int quantita) {  
    return prezzo * quantita;  
}
```

Qui il metodo restituisce un `double` .

Puoi usare il risultato così:

```
double totale = calcolaTotale(2.50, 4);  
System.out.println(totale);
```

Output:

10.0

Il tipo di ritorno deve corrispondere

Se dichiari che un metodo restituisce `int` , deve restituire un `int` .

```
public static int doppio(int numero) {  
    return numero * 2;  
}
```

Questo invece è sbagliato:

```
public static int saluto() {  
    return "Ciao"; // errore  
}
```

Il metodo promette un `int` , ma prova a restituire una `String` .

Esempio completo

```
public class Calcoli {  
    public static double calcolaTotale(double prezzo, int quantita) {  
        return prezzo * quantita;  
    }  
  
    public static void mostraTotale(String prodotto, double totale) {  
        System.out.println(prodotto + ": " + totale + " euro");  
    }  
  
    public static void main(String[] args) {  
        double totalePane = calcolaTotale(1.50, 2);  
        double totaleLatte = calcolaTotale(1.20, 3);  
  
        mostraTotale("Pane", totalePane);  
        mostraTotale("Latte", totaleLatte);  
    }  
}
```

Output:

```
Pane: 3.0 euro  
Latte: 3.5999999999999996 euro
```

Il secondo numero mostra una piccola imprecisione tipica dei numeri decimali nei computer. Per ora ci basta sapere che puo succedere.

Regola pratica

Usa parametri quando il metodo ha bisogno di dati dall'esterno.

Usa `return` quando il metodo deve calcolare o preparare un valore da usare dopo.

Scope

Dove esistono le variabili e perche alcune non sono visibili ovunque.

Cosa significa scope

Lo **scope** è la zona del programma in cui una variabile esiste ed è visibile.

Una variabile non vive ovunque. Di solito vive solo dentro il blocco in cui è stata creata.

Variabili locali

Una variabile creata dentro un metodo si chiama variabile locale.

```
public static void saluta() {  
    String nome = "Luca";  
    System.out.println(nome);  
}
```

`nome` esiste solo dentro `saluta` .

Questo codice è sbagliato:

```
public static void saluta() {  
    String nome = "Luca";  
}  
  
public static void main(String[] args) {  
    System.out.println(nome); // errore  
}
```

`main` non vede la variabile `nome` .

Parametri

Anche i parametri esistono solo dentro il metodo.

```
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome);  
}
```

`nome` è visibile dentro `saluta`, ma non fuori.

Se vuoi usare un valore in un metodo, passalo come parametro.

Blocchi con parentesi graffe

Anche un blocco `if` o `for` crea una zona.

```
if (true) {  
    int numero = 10;  
    System.out.println(numero);  
}  
  
// System.out.println(numero); // errore
```

`numero` nasce dentro il blocco `if` e sparisce alla fine del blocco.

Variabili create nel for

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}  
  
// System.out.println(i); // errore
```

La variabile `i` esiste solo dentro il ciclo.

Questo è utile: evita che contatori temporanei restino in giro nel resto del programma.

Due variabili con lo stesso nome

Non puoi dichiarare due variabili con lo stesso nome nello stesso blocco.

```
int eta = 20;  
int eta = 21; // errore
```

Puoi invece modificare la variabile:

```
int eta = 20;  
eta = 21;
```

Campi di classe

Piu avanti useremo variabili dichiarate dentro una classe, ma fuori dai metodi. Si chiamano **campi**.

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Questi campi appartengono agli oggetti della classe `Persona`.

Per ora ti basta distinguere:

- variabili locali: dentro un metodo o blocco
- parametri: valori ricevuti da un metodo
- campi: dati di un oggetto

Esempio corretto con parametri

Se un metodo deve usare un valore, passalo:

```
public class EsempioScope {  
    public static void saluta(String nome) {  
        System.out.println("Ciao, " + nome);  
    }  
  
    public static void main(String[] args) {  
        String nomeUtente = "Sara";  
        saluta(nomeUtente);  
    }  
}
```

Output:

```
Ciao, Sara
```

`nomeUtente` esiste in `main`. Il suo valore viene passato a `saluta`, dove arriva nel parametro `nome`.

Regola pratica

Quando Java dice che una variabile "cannot be resolved", spesso significa che stai provando a usarla fuori dal suo scope.

Controlla dove è stata dichiarata e dentro quali parentesi graffe si trova.